UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA
Desarrollo Web
Ing. MGSI. César Hernández

Guali

Práctica de programación introductoria en línea

Proyecto de Curso - Fase 2

Documentación técnica

Estudiante: [NOMBRE COMPLETO DEL ESTUDIANTE]

Carné: [####-##-###]

Versión 2.0.0 | Fecha de entrega: 7 de noviembre de 2026

1. Control de versiones

Versión	Descripción
1.0.0	Fase 1: interfaz gráfica general, menús, apertura de lecciones, configuración en memoria.
2.0.0	Fase 2: persistencia en MariaDB, seguridad y administración, compilar y ejecutar con guali-dev, historial, importar/exportar lecciones, documentación técnica actualizada.

2. Resumen del sistema

Guali es una aplicación web para practicar programación Java con lecciones cortas. El estudiante abre una lección, escribe el código manualmente y lo compila y ejecuta desde el navegador, sin herramientas externas de asistencia. No necesita autenticarse. El administrador inicia sesión para gestionar lecciones, usuarios administradores y ver estadísticas de uso.

Este documento describe el sistema **tal como está implementado** en la Fase 2: arquitectura, base de datos, despliegue, decisiones de diseño y el grado de cumplimiento del alcance. Las partes que no se completaron se indican de forma explícita en las secciones 9 y 10.

2.1 Tecnologías y versiones

Componente	Tecnología / versión	Uso
Lenguaje y plataforma	Java 25 (JDK completo)	Aplicación y compilación de código de estudiantes
Framework	Spring Boot 3.5.3	Servidor web embebido (Tomcat), inyección de dependencias
Patrón MVC	Spring MVC + Thymeleaf	Controladores y vistas del lado servidor
API REST	Spring Web (@RestController)	/api/compilar, /api/ejecutar, exportación
Persistencia	Spring Data JPA / Hibernate	Entidades y repositorios
Base de datos	MariaDB 11.8 (puerto 3307)	Lecciones, usuarios administradores, historial
Seguridad	Spring Security	Autenticación, roles, BCrypt, CSRF
Compilador	guali-dev.jar 1.0.0 (provisto por el catedrático)	Compila y ejecuta código Java en tiempo real
Archivos zip	Apache Commons Compress 1.26.2	Importar y exportar lecciones
Frontend	HTML, CSS y JavaScript	Interfaz estática (Fase 1) y plantillas Thymeleaf

3. Arquitectura

La aplicación es un **monolito modular** de Spring Boot organizado por capas. Una sola aplicación atiende las páginas, la API REST, la seguridad y el acceso a datos, y consume guali-dev.jar como biblioteca.

Capa / paquete	Contenido	Responsabilidad
Controller	LeccionController, HistorialController, CompilarController, LoginController, AdminController, AdminUsuarioController, AdminLeccionController, AdminEstadisticasController, ImportarLeccionesController, ExportarLeccionesController, AdminActualAdvice	Reciben las peticiones, llaman a los repositorios o a guali-dev y eligen la vista o la respuesta JSON.

Capa / paquete	Contenido	Responsabilidad
entity	Leccion, UsuarioAdmin, IntentoHistorial	Mapeo JPA de las tres tablas.
Repository	LeccionRepository, UsuarioAdminRepository, IntentoHistorialRepository	Acceso a MariaDB con Spring Data JPA.
Seguridad	SecurityConfig, MigradorContrasenas	Reglas de acceso, login/logout, BCrypt, migración de contraseñas.
templates	lecciones, historial, login, admin_*, form_usuario, form_leccion	Vistas Thymeleaf.
static	index.html, css, js, lecciones/	Interfaz estática de la Fase 1 (tema y fuente en memoria).

3.1 Flujo de compilar y ejecutar

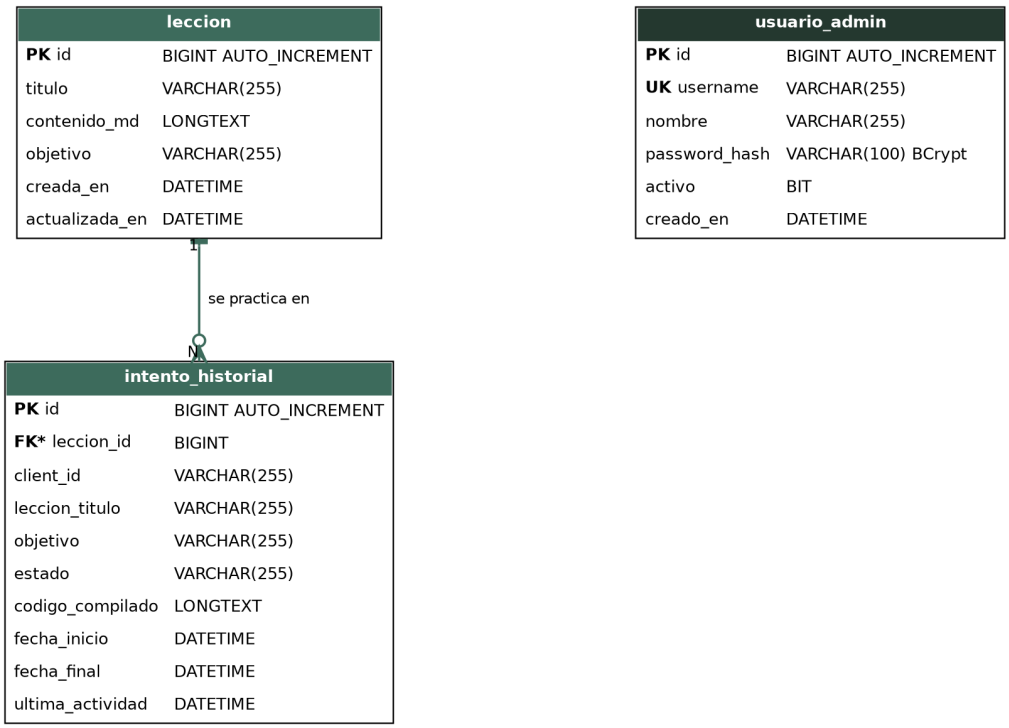
- El navegador envía el código con `POST /api/compilar` o `/api/ejecutar` (JSON).
- **CompilarController** llama a `GualiDev.compile(codigo, jdkHome)`. Si hay errores, devuelve el texto del compilador.
- Para ejecutar, si la compilación fue exitosa llama a `GualiDev.execute(nombreClase, jdkHome)` y devuelve la salida estándar, los errores o un aviso de tiempo agotado.
- La ruta del JDK se configura con la propiedad `guali.jdk-home` (por defecto, el JDK con el que corre la aplicación).

3.2 Flujo de autenticación

- Cualquier ruta bajo `/admin/**` exige el rol ADMIN. Sin sesión, el usuario es redirigido a `/login`.
- `UserDetailsService` busca al usuario en **usuario_admin**; la contraseña se compara con BCrypt y un usuario con activo = falso no puede entrar.
- Login correcto: redirección a `/admin/panel`. Login incorrecto: `/login?error`. Cerrar sesión (POST con token CSRF): vuelve a `/lecciones`.

4. Diagrama Entidad-Relación

El modelo tiene tres tablas, correspondientes a las tres entidades JPA. El script `docs/schema.sql` contiene su definición completa.



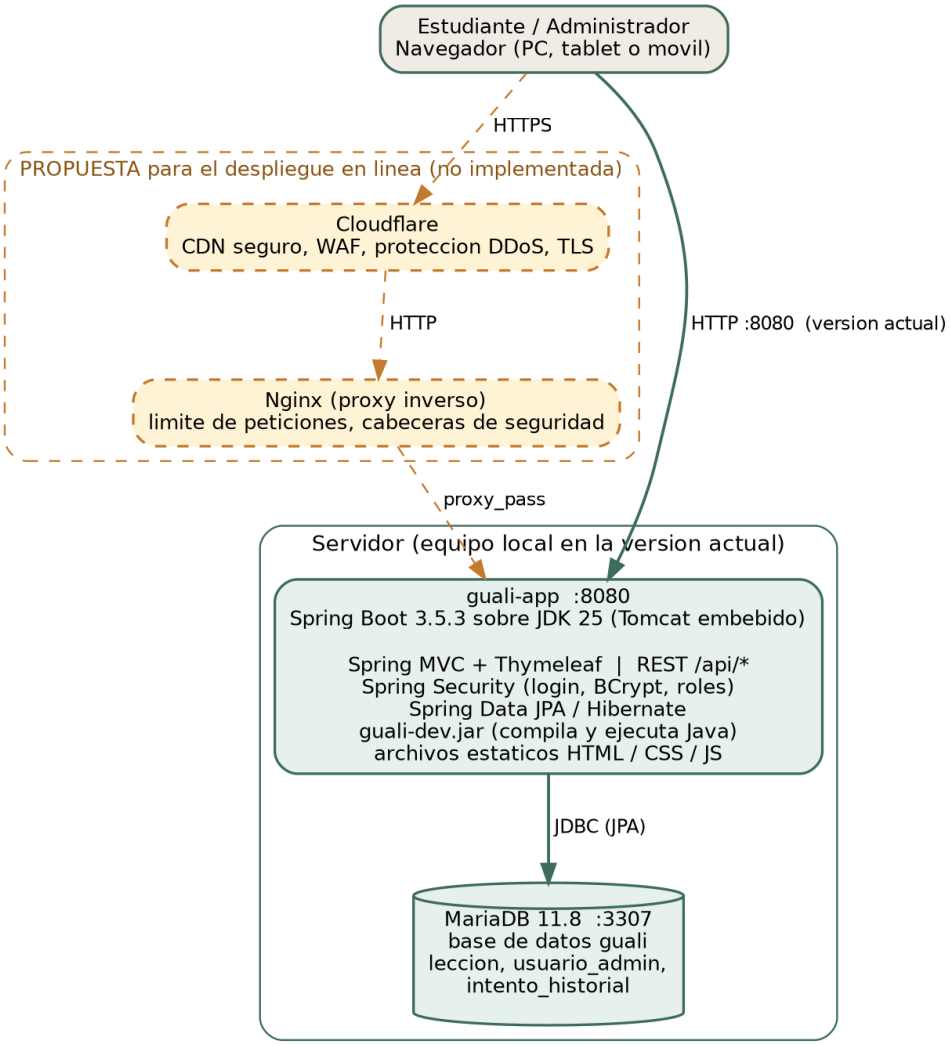
* La relación entre intento_historial.leccion_id y leccion.id es de uno a muchos. En el código es una referencia por columna (no una asociación JPA); si la base de datos tiene la llave foránea declarada, MariaDB impide eliminar una lección con historial. El historial conserva también leccion_titulo para poder mostrarse aunque la lección cambie.

4.1 Diccionario de datos

Tabla	Campo	Descripción
leccion	titulo	Nombre de la carpeta de la lección (p. ej. leccion-01-hola-mundo).
	contenido_md	Texto de la lección en Markdown (LONGTEXT).
	objetivo	COMPILAR_CON_ERROR, COMPILAR_EXITOSO, EJECUTAR_CON_ERROR o EJECUTAR_EXITOSO.
usuario_admin	username / nombre	Identificador de acceso y nombre completo del administrador.
	password_hash	Contraseña con BCrypt (60 caracteres). Nunca en texto plano.
	activo	Si es falso, el usuario no puede iniciar sesión.
intento_historial	client_id	Identifica al estudiante. Actualmente el valor fijo 'estudiante-web'.
	estado	PENDIENTE al abrir la lección; COMPLETADA cuando se cumple el objetivo (ver sección 9).
	fecha_inicio / fecha_final	Apertura de la lección y cierre (completada, abandonada o cancelada).
	ultima_actividad / codigo_compilado	Última interacción y código enviado a compilar.

5. Diagrama de despliegue

En la versión actual la aplicación y la base de datos corren en el mismo equipo. En naranja se muestra la **propuesta** para el despliegue en línea con las medidas de seguridad de borde recomendadas en el enunciado (WAF, protección contra denegación de servicio, CDN seguro y proxy inverso con seguridad); esas piezas **no están implementadas**.



5.1 URL del proyecto desplegado

[URL DEL PROYECTO DESPLEGADO - PENDIENTE]

El proyecto todavía no está publicado en línea. Esta sección debe completarse con la dirección pública una vez desplegado.

6. Mapa de rutas y control de acceso

Ruta	Método	Acceso	Función
/	GET	Público	Interfaz estática de la Fase 1 (index.html).
/lecciones	GET	Público	Lista de lecciones y editor.
/lecciones/{id}	GET	Público	Abre la lección y registra un intento en el historial.
/historial	GET	Público	Panel de historial (más reciente primero).
/api/compilar	POST	Público	Compila con guali-dev. Respuesta JSON: exito, salida.
/api/ejecutar	POST	Público	Compila y ejecuta. Respuesta JSON: salida.
/login	GET / POST	Público	Formulario y autenticación del administrador.
/logout	POST	Sesión	Cierra la sesión (token CSRF).
/admin, /admin/panel	GET	ADMIN	Panel de administración.
/admin/usuarios	GET	ADMIN	Lista de administradores.
/admin/usuarios/nuevo	GET	ADMIN	Formulario de nuevo administrador.
/admin/usuarios/guardar	POST	ADMIN	Crea o actualiza un administrador (BCrypt).
/admin/usuarios/eliminar/{id}	POST	ADMIN	Elimina un administrador.
/admin/lecciones	GET	ADMIN	Lista de lecciones, importar y exportar.
/admin/lecciones/nuevo, /guardar	GET / POST	ADMIN	Crear una lección.
/admin/lecciones/eliminar/{id}	POST	ADMIN	Elimina una lección.
/admin/importar/lecciones	POST	ADMIN	Importa un compendio .zip.
/admin/exportar/lecciones	GET	ADMIN	Descarga lecciones.zip.
/admin/estadisticas	GET	ADMIN	Totales de uso.

Regla de seguridad: `/admin/**` exige ROLE_ADMIN; todo lo demás es público. CSRF está activo en todo el sitio excepto en `/api/**`.

7. Decisiones importantes y su justificación

D1. Monolito Spring Boot con MVC en lugar de microservicios

El enunciado menciona microservicios, pero el proyecto es individual y de plazo corto, y pide integrar guali-dev.jar como *dependencia* del proyecto. Se optó por una sola aplicación que consume la biblioteca directamente: un solo despliegue, menos piezas que fallar y depuración más sencilla. El costo es que el código de los estudiantes se compila y ejecuta en el mismo servidor que la aplicación; separar el compilador en su propio servicio es la evolución natural (ver sección 9).

D2. Dos interfaces: estática (Fase 1) y dinámica (Fase 2)

La interfaz estática de la Fase 1 (`/`) se conservó sin cambios para no perder el entregable anterior; mantiene el tema y el tamaño de fuente en memoria, como pedía esa fase. El flujo de la Fase 2 (base de datos, compilar, historial, administración) vive en las vistas Thymeleaf bajo `/lecciones` y `/admin`.

D3. JPA con MariaDB y esquema controlado (ddl-auto=none)

Las lecciones y su historial deben sobrevivir a cada ejecución, por lo que se persisten en MariaDB con Spring Data JPA. Hibernate no crea ni modifica tablas (`ddl-auto=none`): así se evita que genere tipos inadecuados, por ejemplo columnas TINYTEXT que truncan el markdown de una lección. Las columnas de texto largo se declaran LONGTEXT y el esquema se mantiene en `docs/schema.sql`.

D4. Lecciones guardadas en la base de datos

Cada lección es una fila: el título es el nombre de la carpeta, el markdown va en LONGTEXT y el objetivo en una columna. Esto permite administrar lecciones sin tocar archivos del servidor y exportarlas e importarlas como zip con la estructura `carpeta/leccion.md + config.txt` del enunciado.

D5. Historial: un registro por apertura de lección

Al abrir una lección se inserta un intento con estado PENDIENTE y fecha de inicio. Como el estudiante no se autentica y no se implementó una identificación anónima, `client_id` tiene un valor fijo; en consecuencia el historial es compartido por todos los estudiantes. La alternativa recomendada es una cookie anónima por navegador (sección 9).

D6. Compilar y ejecutar como endpoints REST sin estado

`/api/compilar` y `/api/ejecutar` reciben el código y responden JSON. Ejecutar vuelve a compilar el código recibido, lo que evita guardar estado de compilaciones en el servidor a costa de compilar dos veces. El JDK se obtiene de la propiedad `guali.jdk-home`, por lo que el proyecto no depende de una ruta fija de un equipo.

D7. Spring Security con sesión, BCrypt y rol ADMIN

El estudiante no requiere autenticación; el administrador sí. Se usa inicio de sesión por formulario con sesión de 30 minutos (cookie HttpOnly, SameSite=Lax), usuarios en MariaDB y contraseñas con BCrypt. **Todo** `/admin/**` exige el rol ADMIN, incluidos importar, exportar y eliminar lecciones. CSRF permanece activo en los formularios; se excluye solo `/api/**`, que son endpoints públicos, sin sesión de usuario que proteger. Se desactivó la caché de petición para evitar el parámetro `?continue` tras el login.

D8. Migración automática de contraseñas a BCrypt

La versión inicial guardaba contraseñas en texto plano. La clase MigradorContrasenas convierte al arrancar cualquier contraseña que no sea un hash BCrypt, de forma idempotente. Así no quedan contraseñas legibles en la base y el administrador conserva la suya.

D9. Importación y exportación de lecciones con zip

La importación guarda el archivo en un temporal, lee solo `leccion.md` y `config.txt` de cada carpeta y borra el temporal al terminar. La exportación genera un zip con una carpeta por lección. Ambas operaciones requieren ADMIN. Limitaciones conocidas en la sección 9.

D10. Administración de usuarios y estadísticas

El módulo de usuarios ofrece listar, crear y eliminar administradores con contraseña BCrypt y bandera activo. El módulo de estadísticas muestra totales (lecciones, administradores e intentos) calculados con consultas de conteo; se eligió esta forma por simplicidad. El conteo de visitantes por intervalo de tiempo no está implementado (sección 9).

D11. Seguridad de borde: propuesta, no implementación

WAF, protección DDoS, CDN seguro y proxy inverso se proponen con Cloudflare delante de Nginx (sección 5): cubren los cuatro elementos con configuración y no con código. No se implementaron porque requieren un servidor con dominio, y el proyecto aún no está desplegado.

8. Cumplimiento del alcance de la Fase 2

Cada fila corresponde a un punto del alcance de la Fase 2 y a los requisitos no funcionales y de documentación del enunciado.

#	Requisito de la Fase 2	Estado	Evidencia / observación
1	Persistencia de la información en base de datos	Parcial	Lecciones, administradores e historial en MariaDB. La configuración (tema, fuente) y las visitas no se persisten.
2	Estudiante sin autenticación	Cumple	Todo el sitio, salvo /admin/**, es público.
3	Administrador con autenticación y autorización	Cumple	Login por formulario, BCrypt, rol ADMIN en /admin/**, CSRF.
4	WAF, protección DoS, CDN seguro, proxy inverso	No implementado	Solo propuesta (sección 5).
5	CRUD de usuarios administradores	Parcial	Crear, listar y eliminar con interfaz. Modificar existe en el controlador pero la interfaz no lo ofrece.
6	Estadísticas de visitas en un intervalo de tiempo	No implementado	Solo totales de lecciones, administradores e intentos.
7	Botón Compilar funciona	Cumple	POST /api/compilar con guali-dev; muestra errores o éxito en la consola.
8	Botón Ejecutar funciona	Parcial	Compila y ejecuta con guali-dev. No exige una compilación exitosa previa (el botón siempre está habilitado).
9	Historial funciona y despliega el panel	Parcial	Panel /historial persistente con lección, fechas y estado. El estado nunca pasa a Completada (no se evalúa el objetivo) y el historial es compartido.
10	Importar, exportar y eliminar lecciones (solo administrador)	Parcial	Importar/exportar zip y eliminar, solo ADMIN. Sin imágenes; viven en el panel de administración, no en el menú Configuración.
11	Configuración: tamaño de fuente y tema	Parcial	Funciona en la interfaz estática (memoria). No está en /lecciones ni se guarda en la base de datos.
12	Arquitectura: HTML, CSS, JS, Spring, MVC, REST, Security, JPA, MariaDB	Cumple	Todas presentes.
13	Arquitectura: microservicios	No implementado	Monolito (decisión D1).
14	Documentación: diagrama E-R actualizado	Cumple	Sección 4.
15	Documentación: diagrama de despliegue	Cumple	Sección 5 (versión actual y propuesta).
16	Documentación: URL del proyecto desplegado	No implementado	Pendiente de publicar el sitio.
17	Documentación: decisiones justificadas	Cumple	Sección 7.

Resumen: **7 cumplen**, **6 parciales**, **4 no implementados**, de 17 puntos.

9. Limitaciones conocidas y trabajo pendiente

Ordenado por impacto sobre el alcance de la Fase 2:

- **Objetivo de la lección no evaluado.** Ningún código marca el intento como COMPLETADA ni registra fecha_final. Hay que evaluar el objetivo (COMPILAR_CON_ERROR, COMPILAR_EXITOSO, EJECUTAR_CON_ERROR, EJECUTAR_EXITOSO) tras compilar o ejecutar y actualizar el intento.
- **Historial compartido.** client_id es fijo. Usar una cookie anónima por navegador permitiría un historial por estudiante.
- **Ejecutar sin compilación previa.** El enunciado indica que Ejecutar solo se habilita si la compilación más reciente fue exitosa.
- **Estadísticas por intervalo.** Requiere una tabla de visitas (un registro por navegador y día) y filtros de fecha.
- **Configuración persistente.** Llevar tema y tamaño de fuente a /lecciones y guardarlos en la base de datos.
- **Lecciones con imágenes.** No hay tabla de imágenes; la importación y exportación las ignoran. La importación tampoco valida el objetivo ni evita duplicados (cada importación inserta de nuevo las lecciones).
- **Administradores.** Falta la edición en la interfaz, impedir que un administrador se elimine a sí mismo o elimine al último, y una política de contraseñas.
- **Aislamiento del compilador.** El código de los estudiantes se ejecuta en el servidor de la aplicación; conviene un servicio o contenedor aparte con límites de recursos y limitar la frecuencia de /api/compilar y /api/ejecutar.
- **Despliegue en línea** con la seguridad de borde propuesta en la sección 5.
- **Credenciales.** La conexión a la base de datos está en application.properties; en producción deben venir de variables de entorno.
- **Eliminar lecciones.** Si la lección tiene historial y la base tiene llave foránea, la eliminación falla y la interfaz no muestra el motivo.

10. Instalación y ejecución

Requisitos: JDK 25 completo (con javac), Maven 3.9 o superior, MariaDB 11.x y el archivo guali-dev.jar entregado por el catedrático.

1. Crear la base de datos. Como usuario root de MariaDB (puerto 3307), ejecutar el script del proyecto y crear el usuario de la aplicación:

```
mariadb -h 127.0.0.1 -P 3307 -u root -p < docs/schema.sql
CREATE USER 'guali'@'localhost' IDENTIFIED BY '';
GRANT ALL PRIVILEGES ON guali.* TO 'guali'@'localhost';
```

Usuario y contraseña deben coincidir con `spring.datasource.*` en application.properties.

2. Instalar guali-dev.jar en el repositorio Maven local:

```
mvn install:install-file -Dfile=guali-dev.jar -DgroupId=edu.umg.guali \
    -DartifactId=guali-dev -Dversion=1.0.0 -Dpackaging=jar
```

3. Crear el administrador inicial (la aplicación convierte la contraseña a BCrypt al arrancar). Hay un INSERT de ejemplo comentado al final de `docs/schema.sql`; cambia la contraseña de ejemplo.

4. Ejecutar desde la carpeta que contiene el pom.xml:

```
mvn clean spring-boot:run
```

5. Abrir `http://localhost:8080/lecciones` (estudiante) o `http://localhost:8080/login` (administrador). Si el JDK no es el que ejecuta Maven, definir `guali.jdk-home` en application.properties.

11. Plan de pruebas manuales

Casos para verificar el sistema. La columna *Resultado obtenido* se completa al ejecutarlos, junto con las capturas de pantalla correspondientes.

#	Caso de prueba	Resultado esperado	Resultado obtenido
P1	Abrir una lección desde el selector.	Se muestra su título, objetivo y contenido; aparece un intento PENDIENTE en /historial.	
P2	Compilar código correcto.	La consola muestra la compilación exitosa.	
P3	Compilar código con un error de sintaxis.	La consola muestra el error del compilador.	
P4	Ejecutar un programa que imprime texto.	La consola muestra la salida del programa.	
P5	Abrir /admin/panel sin sesión.	Redirección a /login.	
P6	Abrir /admin/lecciones, /admin/exportar/lecciones y /admin/estadisticas sin sesión.	Redirección a /login en los tres casos.	
P7	Iniciar sesión con contraseña incorrecta.	Vuelve a /login?error con el mensaje de error.	
P8	Iniciar sesión con credenciales correctas.	Llega a /admin/panel.	
P9	Crear un administrador y entrar con él.	Aparece en la lista y puede iniciar sesión.	
P10	Importar un .zip con lecciones.	Las lecciones aparecen en la lista y en el selector.	

#	Caso de prueba	Resultado esperado	Resultado obtenido
P11	Exportar lecciones.	Se descarga lecciones.zip con una carpeta por lección.	
P12	Crear una lección nueva desde el formulario.	Aparece en la lista de lecciones.	
P13	Cerrar sesión.	Redirección a /lecciones; /admin/panel vuelve a pedir login.	